

第三十一讲：Buffer Pool 的 LRU 页面置换算法

知春路遇上八里桥

<2026-08-17 Mon>



- 1 LRU 页面置换算法
- 2 LRU 三个动作的实现
- 3 页面淘汰与回收
- 4 淘汰路径的分支与辅助机制
- 5 页面访问并发控制
- 6 小结



1

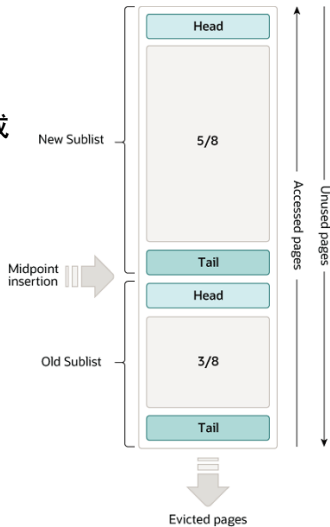
LRU 页面置换算法



LRU 功能设计

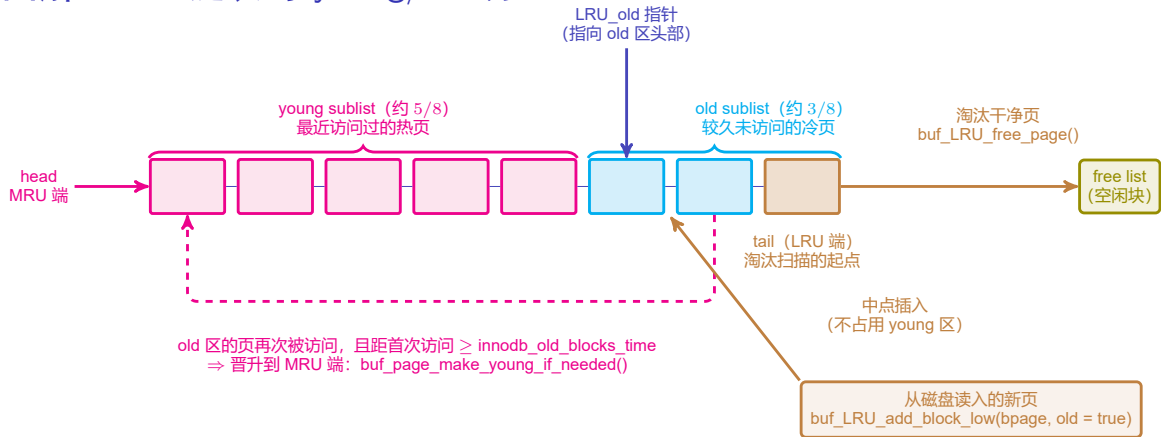
- 1 InnoDB 采用的是改良的 LRU 算法¹
- 2 当 LRU 列表长度达到 BUF_LRU_OLD_MIN_LEN (512) 时会分成
 - ▶ Young SubList 存储最近访问过的页（热数据）
 - ▶ Old SubList 存储较久未访问的页（冷数据）
- 3 从头部开始大概占据 5/8 长度的是 Young SubList, 剩余的是 Old SubList
- 4 防止全表扫描等操作污染整个 LRU 列表
 - ▶ mysqldump 工具全表扫描
 - ▶ 全表扫描的 SELECT 语句

```
Pending writes: LRU 0, flush list 0, single page 0
Pages made young 4, not young 0
0.10 youngs/s, 0.00 non-youngs/s
Pages read 197, created 5523, written 5060
0.00 reads/s, 190.89 creates/s, 244.94 writes/s
Buffer pool hit rate 1000 / 1000, young-making rate 0 / 1000 not
0 / 1000
Pages read ahead 0.00/s, evicted without access 0.00/s, Random read
ahead 0.00/s
LRU len: 5720, unzip_LRU len: 0
I/O sum[0]:cur[0], unzip sum[0]:cur[0]
```



¹<https://dev.mysql.com/doc/refman/8.0/en/innodb-buffer-pool.html>

图解: LRU 链表的 young/old 分区



- 图中标注四个动作: 新页中点插入至 old 区头部、再次访问后晋升至 MRU 端、自链表尾部选取可替换页、LRU_old 指针维护 young (约 5/8) 与 old (约 3/8) 的分界
- 后续各页依次给出这四个动作的实现代码

LRU 算法实现总览

- 页的插入：新页采用中点插入，置于 old 区头部而非 MRU 端
 - ▶ `buf_LRU_add_block_low(bpage, old)`，实现见“新页中点插入”一节
 - ▶ 目的是避免全表扫描产生的一次性页驱逐 young 区中的页
- 页的晋升：页在 old 区被再次访问，且距首次访问超过阈值才升到 young 区
 - ▶ `buf_page_make_young_if_needed()` -> `buf_page_peek_if_too_old()`
 - ▶ `buf_page_peek_if_young()` 是它的回退启发式，全库仅有这一个调用点
- 分界的维护：`buf_LRU_old_adjust_len()` 以容忍带抑制分界抖动
- 页的替换：Buffer Pool 无空闲块时，从 LRU 尾部选取可替换页
 - ▶ 入口 `buf_LRU_get_free_block()`，以 `goto loop` 迭代逐级升级手段（非递归实现）
- LRU_old 指针指向分界点，见 `include/buf0buf.h` 第 2360 行

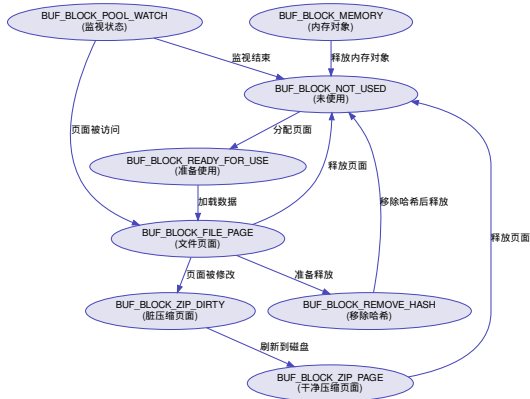
```
2360  /** Pointer to the about LRU_old_ratio/BUF_LRU_OLD_RATIO_DIV oldest blocks in
2361  the LRU list; NULL if LRU length less than BUF_LRU_OLD_MIN_LEN; NOTE: when
2362  LRU_old != NULL, its length should always equal LRU_old_len */
2363  buf_page_t *LRU_old;
```



页面状态机

storage/innobase/include/buf0buf.h

```
124  /** @brief States of a control block
125  @see buf_page_t
126
127  The enumeration values must be 0..7. */
128  enum buf_page_state : uint8_t {
129  /** A sentinel for the buffer pool watch, element of buf_pool->watch[] */
130  BUF_BLOCK_POOL_WATCH,
131  /** Contains a clean compressed page */
132  BUF_BLOCK_ZIP_PAGE,
133  /** Contains a compressed page that is in the buf_pool->flush_list */
134  BUF_BLOCK_ZIP_DIRTY,
135
136  /** Is in the free list; must be after the BUF_BLOCK_ZIP_ constants for
137  compressed-only pages @see buf_block_state_valid() */
138  BUF_BLOCK_NOT_USED,
139
140  /** When buf_LRU_get_free_block returns a block, it is in this state */
141  BUF_BLOCK_READY_FOR_USE,
142
143  /** Contains a buffered file page */
144  BUF_BLOCK_FILE_PAGE,
145
146  /** Contains some main memory object */
147  BUF_BLOCK_MEMORY,
148
149  /** Hash index should be removed before putting to the free list */
150  BUF_BLOCK_REMOVE_HASH,
151  };
```



LRU 分区与调整相关的关键常量

- 见 `storage/innobase/include/buf0lru.h` 与 `storage/innobase/buf/buf0lru.cc`

```
/* buf0lru.h */
constexpr uint32_t BUF_LRU_OLD_MIN_LEN      = 8 * 1024 / 16; // 59: = 512
constexpr uint32_t BUF_LRU_OLD_RATIO_DIV    = 1024;          // 217: ratio 的分母
constexpr uint32_t BUF_LRU_OLD_RATIO_MAX    = BUF_LRU_OLD_RATIO_DIV; // 221
constexpr uint32_t BUF_LRU_OLD_RATIO_MIN    = 51;            // 227: ratio 下限
/* buf0lru.cc */
constexpr uint32_t BUF_LRU_OLD_TOLERANCE    = 20;            // 67: 容忍偏差
constexpr uint32_t BUF_LRU_NON_OLD_MIN_LEN  = 5;             // 72: young 区下限
```

- BUF_LRU_OLD_MIN_LEN (512): 即 8MB / 16KB。LRU 链表长度达到此值才启用 old/young 分区, 否则所有页一律插到 MRU 端、无 LRU_old 指针 (见 `buf_LRU_old_init()`, `buf0lru.cc` 第 1513 行)
- LRU_old_ratio 默认值由 `buf_LRU_old_ratio_update(100 * 3 / 8, false)` 设置 (`storage/innobase/buf/buf0lru.cc` 第 1527 行)
 - 注意 $100 * 3 / 8$ 是整数除法, 得 37 而不是 37.5
 - 转换公式 $\text{ratio} = \text{old_pct} * \text{BUF_LRU_OLD_RATIO_DIV} / 100$, 即 $37 * 1024 / 100 = 378$
- 故 old 区约占 $378/1024 \approx 3/8$, young 区约占 $5/8$



2

LRU 三个动作的实现



算法细节：新页中点插入 (Midpoint Insertion)

- 从磁盘读入的数据页默认以 `old=true` 调用 `buf_LRU_add_block_low()`，插入到 `LRU_old` 指针之后（即 `old` 区头部），而非 `MRU` 端，避免全表扫描污染 `young` 区

▶ 见 `storage/innobase/buf/buf0lru.cc` 第 1663 行（精简代码）

```
1663 static inline void buf_LRU_add_block_low(buf_page_t *bpage, bool old) {
1664     buf_pool_t *buf_pool = buf_pool_from_bpage(bpage);
1665     ut_ad(mutex_own(&buf_pool->LRU_list_mutex));
1666
1667     if (!old || (UT_LIST_GET_LEN(buf_pool->LRU) < BUF_LRU_OLD_MIN_LEN)) {
1668         UT_LIST_ADD_FIRST(buf_pool->LRU, bpage);           // 插到 MRU 端
1669         bpage->freed_page_clock = buf_pool->freed_page_clock;
1670     } else {
1671         UT_LIST_INSERT_AFTER(buf_pool->LRU, bpage, buf_pool->LRU_old);
1672         buf_pool->LRU_old_len++;                             // old 区头
1673     }
1674
1675     if (UT_LIST_GET_LEN(buf_pool->LRU) > BUF_LRU_OLD_MIN_LEN) {
1676         buf_page_set_old(bpage, old);
1677         buf_LRU_old_adjust_len(buf_pool);                   // 调整分界
1678     } else if (UT_LIST_GET_LEN(buf_pool->LRU) == BUF_LRU_OLD_MIN_LEN) {
1679         buf_LRU_old_init(buf_pool);                         // 首次建分界
1680     } else {
1681         buf_page_set_old(bpage, buf_pool->LRU_old != nullptr);
1682     }
1683 }
```

- LRU 长度 < 512 时无分区，一律插到 `MRU` 端



算法细节: young 化判断 (第二次访问且超时才晋升)

- 页再次被访问时, `buf_page_make_young_if_needed()` 调用 `buf_page_peek_if_too_old()` 判断是否晋升到 young 区
- `buf_LRU_old_threshold` 为毫秒级阈值 (对应 `innodb_old_blocks_time`), 避免全表扫描的页在读入后短时间内被再次访问即晋升到 young 区
- 回退分支 `buf_page_peek_if_young()` 用 `freed_page_clock` 启发式: 池计数器相对该块入队时的增量小于阈值, 即判定为仍属新近访问、不予晋升

▶ 见 `storage/innobase/include/buf0buf.ic` 第 180 行 (精简代码)

```
180 static inline bool buf_page_peek_if_too_old(const buf_page_t *bpage) {
181     buf_pool_t *buf_pool = buf_pool_from_bpage(bpage);
182     if (buf_pool->freed_page_clock == 0) {
183         return false; // 暖机/纯内存负载, 无驱逐
184     } else if (get_buf_LRU_old_threshold() != std::chrono::seconds::zero()
185                && bpage->old) {
186         const auto t = buf_page_is_accessed(bpage); // 首次访问时间
187         if (t != std::chrono::steady_clock::time_point{} &&
188             std::chrono::steady_clock::now() - t >= get_buf_LRU_old_threshold()) {
189             return true; // 距首次访问超阈值 → 晋升
190         }
191         buf_pool->stat.n_pages_not_made_young++;
192         return false;
193     } else {
194         return (!buf_page_peek_if_young(bpage)); // 回退到 freed_page_clock 启发式
195     }
196 }
```

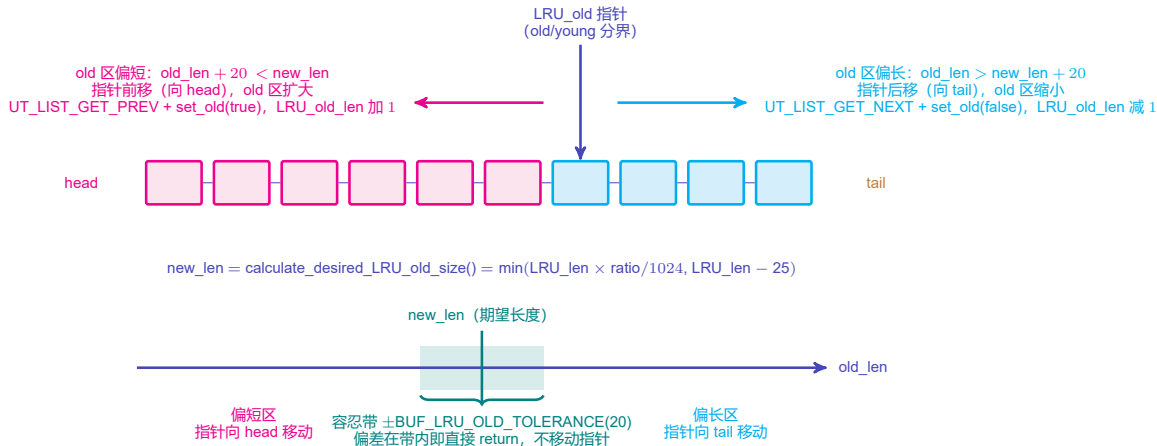


算法细节: old 区长度自调整 (容忍带)

- `buf_LRU_old_adjust_len()` 通过移动 `LRU_old` 指针维护 old/young 分界, 期望长度由 `calculate_desired_LRU_old_size()` 计算: $\min(\text{LRU_len} * \text{ratio}/1024, \text{LRU_len}-25)$
 - `BUF_LRU_OLD_TOLERANCE = 20` 为容忍偏差, 避免每次增删页都调整分界; 调整通过逐块翻转 old 标志实现, 指针向 head 移动则 old 区扩大, 向 tail 移动则 old 区缩小
- ▶ 见 `storage/innobase/buf/buf0lru.cc` 第 1456 行 (精简代码)

```
1456 static inline void buf_LRU_old_adjust_len(buf_pool_t *buf_pool) {
1457     ulint old_len = buf_pool->LRU_old_len;
1458     ulint new_len = calculate_desired_LRU_old_size(buf_pool);
1459     for (;;) {
1460         buf_page_t *LRU_old = buf_pool->LRU_old;
1461         if (old_len + BUF_LRU_OLD_TOLERANCE < new_len) {           // old 区偏短
1462             buf_pool->LRU_old = LRU_old = UT_LIST_GET_PREV(LRU, LRU_old);
1463             old_len = ++buf_pool->LRU_old_len;                     // 指针前移
1464             buf_page_set_old(LRU_old, true);                       // 前一节点并入 old
1465         } else if (old_len > new_len + BUF_LRU_OLD_TOLERANCE) {    // old 区偏长
1466             buf_pool->LRU_old = UT_LIST_GET_NEXT(LRU, LRU_old);   // 指针后移
1467             old_len = --buf_pool->LRU_old_len;
1468             buf_page_set_old(LRU_old, false);                      // 当前节点划入 young
1469         } else {
1470             return;                                                 // 在容忍带 (±20) 内
1471         }
1472     }
1473 }
```

图解: old/young 分界的自调整



- 偏差在容忍带内即直接 return, 避免每次增删页都移动 LRU_old 指针
- 移动指针时逐块翻转 old 标志, 并同步维护 LRU_old_len

常量之间的约束关系

- 三个常量并非各自独立取值，而是相互约束
- BUF_LRU_OLD_TOLERANCE (20): old 区实际长度与期望长度偏差在 ± 20 以内时不移动 LRU_old 指针，避免每次增删页都调整分界
- BUF_LRU_NON_OLD_MIN_LEN (5): young 区至少保留 5 个块。期望 old 区长度取 $\min(\text{LRU_len} * \text{ratio} / 1024, \text{LRU_len} - (20 + 5))$ ，其中第二项保证 young 区非空
 - 即 calculate_desired_LRU_old_size() (☞ [buf0lru.cc](#) 第 1445 行)
- BUF_LRU_OLD_RATIO_MIN (51): ratio 的下限，由前两个常量反推得出
 - 源码以一条 static_assert 固化该约束 (☞ [buf0lru.cc](#) 第 1464 行)

```
1464 static_assert(BUF_LRU_OLD_RATIO_MIN * BUF_LRU_OLD_MIN_LEN >
1465               BUF_LRU_OLD_RATIO_DIV * (BUF_LRU_OLD_TOLERANCE + 5),
1466               "BUF_LRU_OLD_RATIO_MIN * BUF_LRU_OLD_MIN_LEN <= "
1467               "BUF_LRU_OLD_RATIO_DIV * (BUF_LRU_OLD_TOLERANCE + 5)");
```

- 代入得 $51 * 512 = 26112 > 1024 * 25 = 25600$ ，等价形式为 $(\text{TOLERANCE} + 5) * \text{DIV} / \text{MIN_LEN} = 25 * 1024 / 512 = 50 < 51$
- 目的：即使 LRU 长度取最小值 512，LRU_old 指针也不会落到链表两端

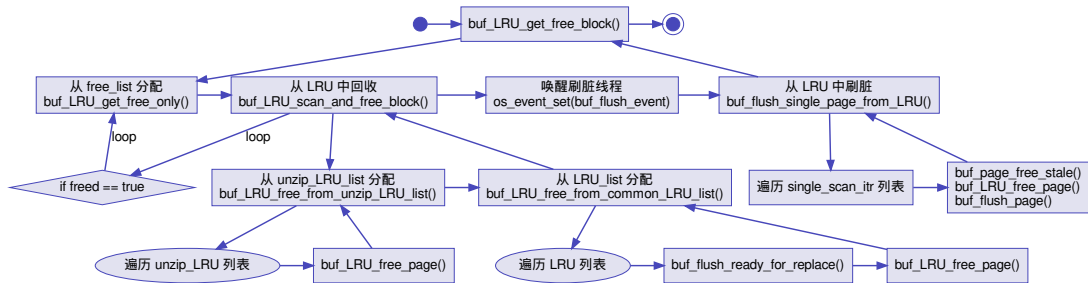


3

页面淘汰与回收



图解：淘汰路径的调用栈



- 主干: `buf_LRU_get_free_block()` → `buf_LRU_scan_and_free_block()` → `buf_LRU_free_from_common_LRU_list()` → `buf_flush_ready_for_replace()` → `buf_LRU_free_page()`
- 本章按该调用栈自上而下展开，压缩页与 stale 页的分支见下一章

空闲块申请流程: buf_LRU_get_free_block()

- 用户线程需要空闲块时调用 buf_LRU_get_free_block(), 采用 goto 循环迭代

▶ 见 `storage/innobase/buf/buf0lru.cc` 第 1327 行 (精简代码)

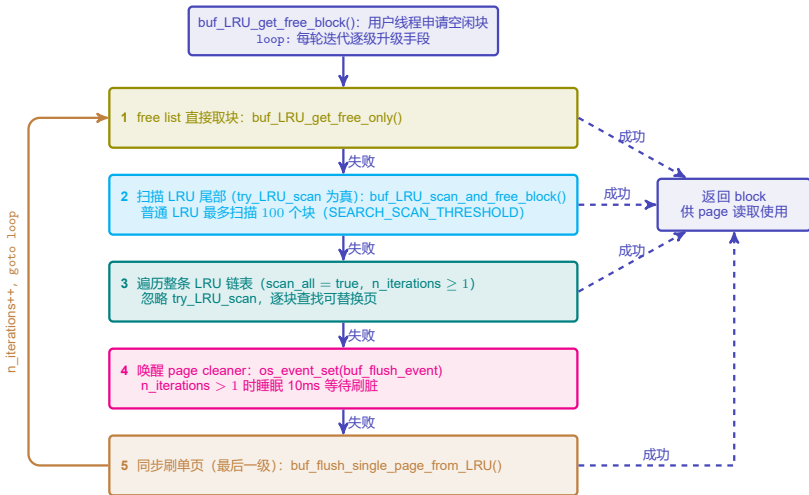
```
1327 buf_block_t *buf_LRU_get_free_block(buf_pool_t *buf_pool) {
1328     loop:
1329     buf_LRU_check_size_of_non_data_objects(buf_pool);
1330     block = buf_LRU_get_free_only(buf_pool);           // 1. free list
1331     if (block != nullptr) { return block; }
1332     if (buf_pool->try_LRU_scan || n_iterations > 0)    // 2. 扫描 LRU
1333         freed = buf_LRU_scan_and_free_block(buf_pool, n_iterations > 0);
1334     if (freed) { goto loop; }
1335     if (!srv_read_only_mode) os_event_set(buf_flush_event); // 3. 唤醒 cleaner
1336     if (n_iterations > 1)                                // 4. 睡眠 10ms
1337         std::this_thread::sleep_for(std::chrono::milliseconds(10));
1338     buf_flush_single_page_from_LRU(buf_pool);          // 5. 同步刷单页
1339     n_iterations++;
1340     goto loop;
1341 }
```

● 迭代策略逐级升级

- ① 第 0 次迭代: 需 try_LRU_scan 为真, 仅扫描 LRU 尾部有限个块 —— 普通 LRU 扫描 BUF_LRU_SEARCH_SCAN_THRESHOLD (100) 个, unzip_LRU 扫描 srv_LRU_scan_depth 个
- ② 第 1+ 次迭代: 扫描整个 LRU 链表 (scan_all=true), 且忽略 try_LRU_scan
- ③ 迭代超过 20 次仍未获得空闲块时, 输出 ER_IB_MSG_134 警告并启动 InnoDB Monitor



图解：空闲块申请的五级升级



迭代 > 20 次仍未获得空闲块: 输出 ER_IB_MSG_134 警告, 并启动 InnoDB Monitor



LRU 扫描淘汰: buf_LRU_scan_and_free_block()

- buf_LRU_scan_and_free_block() 先尝试 unzip_LRU, 再尝试普通 LRU

▶ 见 `storage/innobase/buf/buf0lru.cc` 第 1157 行

```
1157 bool buf_LRU_scan_and_free_block(buf_pool_t *buf_pool, bool scan_all) {
1158     bool freed = false;
1159     bool use_unzip_list = UT_LIST_GET_LEN(buf_pool->unzip_LRU) > 0;
1160     mutex_enter(&buf_pool->LRU_list_mutex);
1161     if (use_unzip_list) {
1162         freed = buf_LRU_free_from_unzip_LRU_list(buf_pool, scan_all);
1163     }
1164     if (!freed) {
1165         freed = buf_LRU_free_from_common_LRU_list(buf_pool, scan_all);
1166     }
1167     if (!freed) { mutex_exit(&buf_pool->LRU_list_mutex); }
1168     return (freed);
1169 }
```

- buf_LRU_free_from_common_LRU_list() (第 1095 行) 从 LRU 尾部向前扫描

- ▶ scan_all=false 时仅扫描前 100 个块 (BUF_LRU_SEARCH_SCAN_THRESHOLD)
- ▶ 先判 was_stale(): stale 页直接调用 buf_page_free_stale(); 否则持块锁经 buf_flush_ready_for_replace() 判定, 可替换则调用 buf_LRU_free_page()
- ▶ 用 lru_scan_itr 维护扫描位置, 避免每次从头开始
- ▶ 同时统计“读入后从未被访问即被淘汰”的页数 (stat.n_ra_pages_evicted)

- buf_LRU_free_from_unzip_LRU_list() (第 1047 行) 有一处注释与代码不符: 6 注释与

srv_LRU_scan_depth / 2, 实际条件是 scanned < srv_LRU_scan_depth



脏页判定条件

① Buffer Pool 只关心在 buf_block_t 中的一个变量：oldest_modification

- ▶ 表示该页自上次变干净以来最初那次修改所属 mtr 的 **start LSN** (mtr0mtr.cc 中 buf_flush_note_modification(block, m_start_lsn, m_end_lsn, ...), 其中 start_lsn 经 buf_flush_insert_into_flush_list() 写入 oldest_modification, end_lsn 写入 newest_modification)
- ▶ oldest_modification 不为 0 表示该页被修改过 (是脏页), is_dirty() 即 get_oldest_lsn() > 0
- ▶ 需要由 page cleaner 线程写回到磁盘上

② 具体定义见 `storage/innobase/include/buf0buf.h`

```
1572  /** The flush LSN, LSN when this page was written to the redo log. For  
1573  non redo logged pages this is set using: buf_flush_borrow_lsn() */  
1574  lsn_t newest_modification;  
1575  
1576  /** log sequence number of the youngest modification to this block, zero  
1577  if not modified. Protected by block mutex */  
1578  lsn_t oldest_modification;
```

- ▶ 注意 oldest_modification 上方 “youngest modification” 是源码里的一处遗留错误注释，真实语义见访问器 get_oldest_lsn() 的注释：the LSN of the first modification since the last time it was clean



替换判定: buf_flush_ready_for_replace()

- 判断页面是否可被直接替换（淘汰），无需刷脏

▶ 见 `storage/innobase/buf/buf0flu.cc` 第 684 行

```
684 bool buf_flush_ready_for_replace(buf_page_t *bpage) {
685     /* 不能替换被固定的页 */
686     if (!buf_page_can_relocate(bpage)) { return false; }
687     /* stale 页只要未被固定即可替换 */
688     if (bpage->was_stale()) { return true; }
689     /* 干净页（未被修改过）可替换 */
690     return !bpage->is_dirty();
691 }
```

- buf_page_can_relocate() 检查 buf_fix_count == 0 且 io_fix == BUF_IO_NONE
- 脏页返回 false: 淘汰路径不执行刷脏，以免阻塞用户线程



页面释放流程: buf_LRU_free_page()

- buf_LRU_free_page(bpage, zip) 是 LRU 淘汰的核心函数，负责从 LRU 和 page_hash 中移除页面

- ▶ 见 `storage/innobase/buf/buf0lru.cc` 第 1757 行
- ▶ 入参 zip: 为 true 时连页的压缩副本一并释放；调用方须持 LRU_list_mutex + block_mutex
- ▶ 返回 true 时 LRU_list_mutex 已由本函数释放
- ▶ 主要步骤：
 - ① 检查页面是否可重定位 (buf_page_can_relocate(): 未被 buffer-fixed 且未被 I/O-fixed)
 - ② 脏页检查 (三种情形见下一节)
 - ③ 若仅释放解压帧、需保留压缩页，先分配新的 buf_page_t 描述符 b
 - ④ 释放 block_mutex，获取 hash_lock 写锁后重新获取 block_mutex
 - ⑤ 重新检查条件 (换锁后状态可能变化)，此后才将 bpage 拷入 b
 - ⑥ 调用 buf_LRU_block_remove_hashed() 从 LRU 和 page_hash 中移除 (hash_lock 由其内部释放；其返回 false 表示页已在其中释放，本函数随即 return true)
 - ⑦ 先 mutex_exit(LRU_list_mutex)，再丢弃自适应哈希索引 (btr_search_drop_page_hash_index(block, true)，此时块处于 BUF_BLOCK_REMOVE_HASH 私有状态)
 - ⑧ 调用 buf_LRU_block_free_hashed_page() 回收到 free list

脏页在释放路径中的三种情形

- 淘汰路径不执行刷脏，但“脏页一律返回 false”的说法并不严格，取决于本次释放的范围 (☞ [buf0lru.cc](#) 第 1779 行，精简代码)

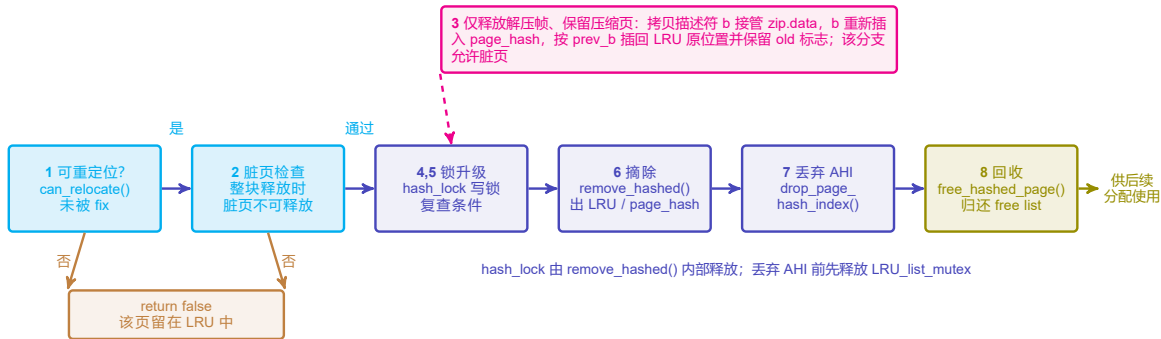
```
1779 if (zip || bpage->zip.data == nullptr) { // 本次会整块释放
1780     if (is_dirty) { return false; }        // 1. 脏页不可释放
1781 } else if (is_dirty &&
1782            buf_page_get_state(bpage) != BUF_BLOCK_FILE_PAGE) {
1783     return false;                          // 2. 脏的纯压缩页 (ZIP_DIRTY) 不可释放
1784 } else if (buf_page_get_state(bpage) == BUF_BLOCK_FILE_PAGE) {
1785     b = buf_page_alloc_descriptor();       // 3. 仅释放解压帧，允许脏页
1786 }
```

- 第 3 种情形 (BUF_BLOCK_FILE_PAGE + 有 zip.data + zip == false) 允许脏页：释放的仅为解压帧，压缩页仍留在 flush_list 中，由 page cleaner 写回
- 两条扫描路径传入的 zip 取值相反

- ▶ 普通 LRU 扫描传 true (第 1123 行)：整块释放，脏页已由 buf_flush_ready_for_replace() 排除
- ▶ unzip_LRU 扫描传 false (第 1071 行)：仅释放解压帧，允许脏页



图解: buf_LRU_free_page() 的处理流程



- 前两步为前置校验：被 fix 的页、以及需整块释放的脏页，均返回 false 并保留在 LRU 中
- 第 4、5 步换锁后必须复查条件：释放 block_mutex 期间页状态可能已发生变化



压缩页的特殊处理：仅释放解压帧

- 若 BUF_BLOCK_FILE_PAGE 且有 zip.data, 可在淘汰解压帧的同时保留压缩页
- new (b) buf_page_t(*bpage) 拷贝的是描述符: zip.data 指针随之转到 b, 随后将 bpage->zip.data 置空、size 改回非压缩
 - ▶ b->state 相应变为 BUF_BLOCK_ZIP_PAGE 或 BUF_BLOCK_ZIP_DIRTY
 - ▶ 脏页经 buf_flush_relocate_on_flush_list(bpage, b) 将 flush_list 中的位置移交给 b
- b 重新插入 page_hash, 并用 UT_LIST_INSERT_AFTER(LRU, prev_b, b) 插回原页在 LRU 中的原位置 (prev_b = UT_LIST_GET_PREV(LRU, b)), 同时保留其 old 标志 (不是统一插到 old 区)
 - ▶ 若 b 位于 old 区, 需 LRU_old_len++, 必要时将 LRU_old 指针改指 b
 - ▶ 随后仍由 buf_LRU_old_adjust_len() / buf_LRU_old_init() 维护分界
- 仅当原页本就位于 MRU 端 (prev_b == nullptr) 时, 才走 buf_LRU_add_block_low(b, buf_page_is_old(b))
- 收尾: buf_page_set_sticky(b) 阻止 buf_page_get_gen() 期间解压该块, 在不持任何 mutex 的条件下计算压缩页 checksum, 随后 buf_page_unset_sticky(b)



4

淘汰路径的分支与辅助机制



stale 页: 表空间版本失效的页面

- stale 页是指 buffer pool 中引用的表空间已被 truncate 或 delete 后遗留的失效页面
- 处理机制: 页面加载时记录表空间版本号, truncate/delete 时 bump_version() 递增版本号使旧页面失效

▶ 见 `storage/innobase/include/buf0buf.h` 第 1208 行 (精简代码)

```
1208 inline bool was_stale() const {  
1209     if (m_version == m_space->get_recent_version()) return false; // 版本一致  
1210     else return true; // 版本不一致, 已失效  
1211 }
```

- 处理: buf_page_free_stale() 直接释放, 不刷脏, 避免旧脏页覆盖新建的 data file



单页同步刷脏: buf_flush_single_page_from_LRU()

- LRU 扫描淘汰失败后，用户线程使用的最后一级手段

- 见 `storage/innobase/buf/buf0flu.cc` 第 2157 行 (精简代码)

```
2157 bool buf_flush_single_page_from_LRU(buf_pool_t *buf_pool) {
2158     mutex_enter(&buf_pool->LRU_list_mutex);
2159     for (bpage = buf_pool->single_scan_itr.start(); bpage != nullptr;
2160          bpage = buf_pool->single_scan_itr.get()) {
2161         if (bpage->was_stale()) {
2162             freed = buf_page_free_stale(buf_pool, bpage);           // stale 页释放
2163         } else {
2164             mutex_enter(block_mutex);
2165             if (buf_flush_ready_for_replace(bpage))
2166                 freed = buf_LRU_free_page(bpage, true);           // 干净页淘汰
2167             else if (buf_flush_ready_for_flush(bpage, BUF_FLUSH_SINGLE_PAGE))
2168                 freed = buf_flush_page(buf_pool, bpage,           // 脏页同步刷
                                         BUF_FLUSH_SINGLE_PAGE, true);
2169         }
2170     }
2171     if (freed) break;
2172 }
2173 if (!freed) mutex_exit(&buf_pool->LRU_list_mutex);
2174 return freed;
2175 }
```

- 使用独立的 `single_scan_itr` 避免与 `lru_scan_itr` 冲突
- 三条路径: stale 页释放 → 干净页淘汰 → 脏页同步刷脏



解压 LRU 淘汰决策: buf_LRU_evict_from_unzip_LRU()

- 当 unzip_LRU 非空时, 决定是淘汰解压帧还是整页淘汰

▶ 见 `storage/innobase/buf/buf0lru.cc` 第 185 行 (精简代码)

```
185 bool buf_LRU_evict_from_unzip_LRU(buf_pool_t *buf_pool) {
186     if (UT_LIST_GET_LEN(buf_pool->unzip_LRU) == 0) return false;
187     if (UT_LIST_GET_LEN(buf_pool->unzip_LRU) <=          // 占比 <= 10% 不淘汰
188         UT_LIST_GET_LEN(buf_pool->LRU) / 10) return false;
189     if (buf_pool->freed_page_clock == 0) return true;      // 暖机阶段
190     ulint io_avg = buf_LRU_stat_sum.io / BUF_LRU_STAT_N_INTERVAL
191                 + buf_LRU_stat_cur.io;
192     ulint unzip_avg = buf_LRU_stat_sum.unzip / BUF_LRU_STAT_N_INTERVAL
193                  + buf_LRU_stat_cur.unzip;
194     return (unzip_avg <= io_avg * BUF_LRU_IO_TO_UNZIP_FACTOR); // I/O 密集
195 }
```

- I/O 密集型负载 ($\text{unzip_avg} \leq \text{io_avg} * 50$): 仅释放解压帧, 保留压缩页
- CPU 密集型负载: 整页淘汰 (必须为干净页), 释放更多内存



LRU 统计信息: buf_LRU_stat_update()

- 每秒由 `srv_error_monitor_thread()` 调用, 维护 50 秒滚动窗口统计

▶ 见 `storage/innobase/buf/buf0lru.cc` 第 2411 行 (精简代码)

```
2411 void buf_LRU_stat_update(void) {
2412     if (!evict_started) goto func_exit;           // 未开始淘汰, 不统计
2413     item = &buf_LRU_stat_arr[buf_LRU_stat_arr_ind]; // 取要被覆盖的最老槽
2414     buf_LRU_stat_arr_ind++;
2415     buf_LRU_stat_arr_ind %= BUF_LRU_STAT_N_INTERVAL; // 环形 50
2416     cur_stat = buf_LRU_stat_cur;                     // 快照 (无锁保护)
2417     buf_LRU_stat_sum.io += cur_stat.io - item->io;    // 滚动窗口: 加新减旧
2418     buf_LRU_stat_sum.unzip += cur_stat.unzip - item->unzip;
2419     memcpy(item, &cur_stat, sizeof *item);
2420     func_exit:
2421     memset(&buf_LRU_stat_cur, 0, sizeof buf_LRU_stat_cur); // 重置本秒计数
2422 }
```

- `buf_LRU_stat_cur` 记录当前秒的 I/O 和解压操作计数
- `buf_LRU_stat_sum` 维护 50 秒滚动窗口的累加值
- `buf_LRU_stat_arr[50]` 环形数组存储各秒的历史值
- `buf_LRU_stat_inc_io()` 和 `buf_LRU_stat_inc_unzip()` 内联函数递增当前计数
(见 `buf0lru.h` 第 258、260 行)



5

页面访问并发控制



LRU 链表的并发保护

- LRU 链表由 `LRU_list_mutex` 保护，所有对 LRU 的增删移动操作都需持有该互斥量
 - ▶ 见 `storage/innobase/include/buf0buf.h` 中 `buf_pool_t` 结构，三把链表锁在结构体中并不相邻（分别位于第 2187、2190、2278 行）

```
BufListMutex LRU_list_mutex;    // 2187: 保护 LRU 链表
BufListMutex free_list_mutex;    // 2190: 保护 free 链表与 withdraw 链表
BufListMutex flush_list_mutex;  // 2278: 保护 flush 链表
```
- 空闲块申请路径中的锁获取顺序
 - ① `buf_LRU_get_free_only()` 仅需 `free_list_mutex`
 - ② `buf_LRU_scan_and_free_block()` 需 `LRU_list_mutex + block_mutex`
 - ③ `buf_flush_single_page_from_LRU()` 需 `LRU_list_mutex + block_mutex`，刷脏时持锁调用 `buf_flush_page()`，由被调方在内部释放这两把锁
- `freed_page_clock` 可在无锁状态下读取：记录 LRU 尾部淘汰的累计次数，用于 young 化启发式判断
 - ▶ `buf_pool_t::freed_page_clock` 与 `buf_page_t::freed_page_clock` 的声明注释均写明 “a thread is allowed to read this for heuristic purposes without holding any mutex or latch”
- `old` 标志按注释归属 LRU 字段组（`LRU_list_mutex + block_mutex` 双重保护），但 `buf_page_peek_if_too_old()` 在只持有 `buf_fix` 的情况下读它 —— 该处仅用于启发式判断，偶发读到旧值不影响正确性（源码未对此写明豁免）

危险指针 (Hazard Pointer) 机制

- LRU 扫描淘汰过程中使用危险指针保护正在访问的节点不被释放

- ▶ 见 `storage/innobase/buf/buf0lru.cc` 第 1558 行

```
1558 void buf_LRU_adjust_hp(buf_pool_t *buf_pool, const buf_page_t *bpage) {
1559     buf_pool->lru_hp.adjust(bpage);
1560     buf_pool->lru_scan_itr.adjust(bpage);
1561     buf_pool->single_scan_itr.adjust(bpage);
1562 }
```

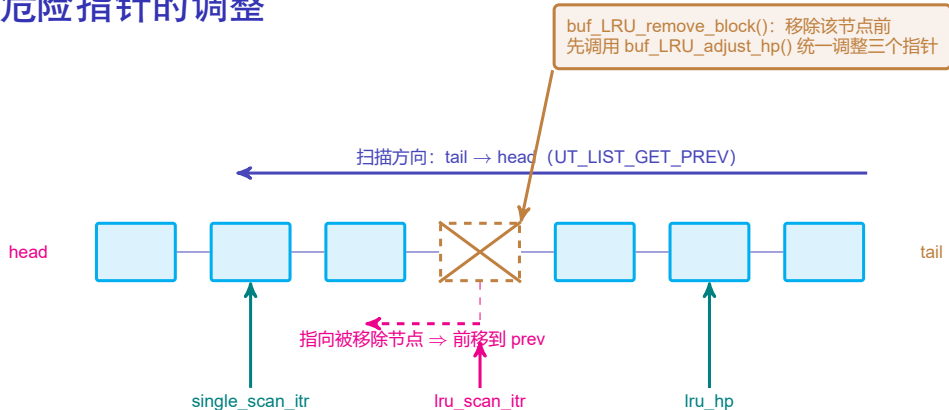
- `buf_pool_t` 中维护三个迭代器/危险指针 (第 2347、2351、2355 行)

- ① `lru_hp`: page cleaner 做 LRU flush batch 扫描时使用 (唯一使用点 `buf_flush_LRU_list_batch()`)
- ② `lru_scan_itr`: `buf_LRU_free_from_common_LRU_list()` 查找可替换页时的扫描迭代器
- ③ `single_scan_itr`: `buf_flush_single_page_from_LRU()` 的扫描迭代器

- `buf_LRU_remove_block()` 是调整方: 移除节点前调用 `buf_LRU_adjust_hp()` 统一调整上述三个指针

- ▶ 若危险指针指向被移除的节点, 则前移到前一个节点, 避免悬空指针
- ▶ `LRUHp::adjust()` 只支持反向遍历 (`m_hp = UT_LIST_GET_PREV(LRU, m_hp)`), 这与 LRU 从尾向头扫描的方向一致

图解：危险指针的调整



危险指针 (Hazard Pointer): buf_pool_t 中的三个扫描迭代器

1. lru_hp: page cleaner 做 LRU flush batch 扫描 (buf_flush_LRU_list_batch)
 2. lru_scan_itr: buf_LRU_free_from_common_LRU_list() 查找可替换页的位置
 3. single_scan_itr: buf_flush_single_page_from_LRU() 的扫描位置
- LRUHp::adjust() 只支持反向遍历 (m_hp = UT_LIST_GET_PREV), 与扫描方向一致

6

小结



LRU 回顾

- **分区**：LRU 长度达到 512 后启用分区，old 区约 3/8 (ratio 378/1024)，young 区约 5/8
 - ▶ 分界由 LRU_old 指针标识，其长度经 ± 20 的容忍带维护，避免抖动
- **入队**：中点插入——自磁盘读入的页插入 old 区头部，不占用 young 区
- **晋升**：old 区的页被再次访问、且距首次访问超过 innodb_old_blocks_time 才升入 young 区
 - ▶ 两个条件共同作用，全表扫描产生的一次性页难以进入 young 区
- **淘汰**：buf_LRU_get_free_block() 五级升级 free list → 扫描 LRU 尾部 100 个块 → 遍历整条 LRU → 唤醒 cleaner 并睡眠 10ms → 同步刷单页
 - ▶ 淘汰路径只选取干净页，buf_flush_ready_for_replace() 对脏页直接返回 false
 - ▶ stale 页（表空间已 drop/truncate）例外：不刷脏，直接释放
- **分支**：压缩页由 buf_LRU_evict_from_unzip_LRU() 单独决策（依据 50 秒滚动窗口统计）
- **并发**：三把链表锁 + 块锁 + 三个危险指针；freed_page_clock / old 等启发式字段有意允许无锁读取
- 下一讲：Change Buffer



结束

